

Audit Feature

Design, implementation and test report — Contracts ERP

Backend: ASP.NET Core 8 / EF Core 8 / MediatR · Database: MySQL 8.0.40 · August 2026

1. The problem this solves

When something goes wrong in the ERP, there is currently no way to answer two questions that always come up first:

- 1. **What was the user actually doing?** — which screen, which action, with what input.
- 2. **What changed in the database, and who changed it?** — which field, from what value to what value.

Three properties of the codebase combine to make this genuinely invisible:

Property	Consequence
Handlers return <code>Result<T></code> instead of throwing	A failed operation looks like an ordinary successful return to everything upstream. No exception, no stack trace, nothing in the error log.
Serilog runs at <code>MinimumLevel.Warning()</code>	Normal activity — the actions that <i>precede</i> a problem — is never written down at all.
Logs are 7-day rolling text files on the VM	Even when something is logged, it cannot be queried, filtered, or joined to the business tables.

This is not hypothetical. The comment block already sitting at the top of `Application/Core/LoggingBehavior.cs` records a real incident: `ResetProjectCertificate` silently orphaned certificate 233-0026's invoices (INV1362 / INV1363) *with zero trace in logs*. That behaviour was added to close half the gap — it logs failed results. This feature closes the other half: what those failures actually did to the data.

2. What was already in the codebase

Because the schema was ported from Apache OFBiz, the audit tables already existed and were already mapped into `DataContext`. They had simply never been connected to anything.

Table	Mapped at	Rows before this work
ENTITY_AUDIT_LOG	<code>DataContext.cs:13888</code>	0
VISIT / SERVER_HIT	<code>DataContext.cs:61715 / 51886</code>	0
USER_LOGIN_HISTORY	<code>DataContext.cs:60908</code>	0

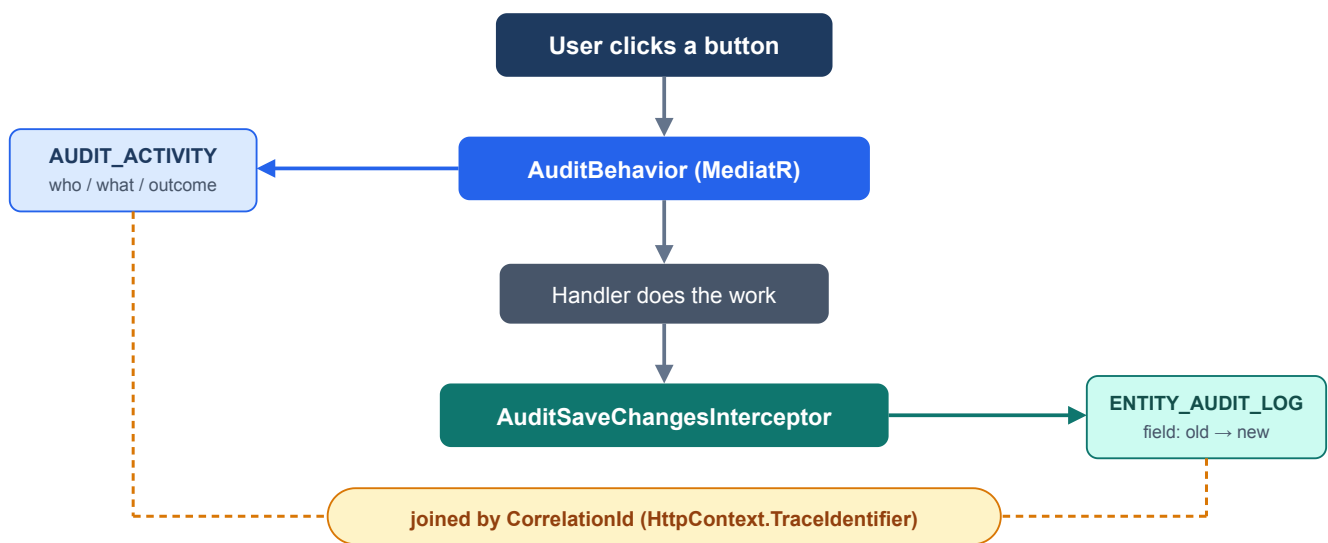
This mattered a great deal for delivery: it meant the first half of the feature could ship **with no database migration at all**, writing into a table that was already there.

3. The design, in plain terms

The feature is built as two independent layers that record different things, tied together by a single shared key.

IN PLAIN TERMS

Think of a warehouse. **Layer A is the door camera** — it records that Emad walked in at 14:32, what he was carrying, and whether he came back out successfully. **Layer B is the stock sheet** — it records that shelf 12 went from 40 units to 38. Neither one is enough on its own. The camera does not tell you what moved; the stock sheet does not tell you who moved it or why. The **correlation ID** is the timestamp that lets you line the two up.



One user action flows through both layers; the correlation ID stitches the two records together.

4. The concepts, explained simply

4.1 A MediatR "pipeline behavior"

Every command in this system travels through MediatR to reach its handler. A *pipeline behavior* is code that wraps every one of those trips — it runs before the handler, calls the handler, and runs again afterwards.

IN PLAIN TERMS

It is a receptionist sitting in front of 610 offices. Every visitor passes the desk on the way in and on the way out, so the receptionist can write down who arrived, what they asked for, how long they took and whether they left happy — without any of the 610 offices needing to change.

The codebase already had one (`LoggingBehavior`), which proved the interception point works. The new `AuditBehavior` is registered right next to it.

4.2 An EF Core "SaveChanges interceptor" and the ChangeTracker

Entity Framework keeps a *ChangeTracker*: an in-memory record of every object it has loaded and what has been altered on it. When `SaveChangesAsync()` is called, EF consults that tracker to work out which SQL statements to emit. An *interceptor* lets us run code at that exact moment.

IN PLAIN TERMS

EF already knows "this invoice's status was **APPROVED** and is about to become **CANCELLED** " — it has to know, in order to write the UPDATE. The interceptor simply asks for that list a fraction of a second before it is thrown away, and writes it down.

This is why the change log can capture the **old** value. Once the UPDATE runs, the previous value is gone forever — the interceptor is the last moment it exists.

4.3 Why one layer is transactional and the other deliberately is not

A database transaction is all-or-nothing: if any part fails, everything inside it is undone.

- **The change log joins the business transaction.** If the operation rolls back, its audit rows roll back too. This is correct — the log must never claim a change that did not actually happen.
- **The activity log deliberately stays outside it**, on its own database connection. If it shared the transaction, a command that failed and rolled back would erase its own audit record — losing exactly the evidence worth having.

WHY THIS IS THE SINGLE MOST IMPORTANT DETAIL

Successes are not the interesting case. The whole feature exists because of an operation that went wrong. If the record of a failure disappears along with the failure, the feature is worthless.

4.4 The correlation ID

ASP.NET assigns every HTTP request a **TraceIdentifier** — a short string like **0HN03HFAAMIPS:00000005** . It was already in use: **ExceptionHandler** returns it to the user as the error ID, and Serilog writes it as **{RequestId}** . Both audit layers now stamp the same value on their rows, so an error a user reports, the log line, the action taken and the rows it changed all line up on one key.

5. Layer 1 — the change log

Records *what changed in the database*, into the pre-existing **ENTITY_AUDIT_LOG** table. No migration required.

5.1 Where the code lives

File	Role
Persistence/Auditing/AuditSaveChangesInterceptor.cs	Reads the ChangeTracker and writes the rows
Persistence/Auditing/AuditedEntities.cs	The whitelist and field-exclusion rules
Persistence/Auditing/IAuditMetadataProvider.cs	Abstraction for "who / which request"
Infrastructure/Security/HttpAuditMetadataProvider.cs	Supplies that from the live HTTP request

5.2 A layering constraint that shaped the design

The project references run *Infrastructure* → *Application* → *Persistence* → *Domain*. The capture code must live in **Persistence** (it needs the ChangeTracker), but the identity of the signed-in user lives in

`Application.Interfaces.IUserAccessor` — which `Persistence` cannot see without creating a circular reference.

Solution: a small interface declared *inside* `Persistence`, implemented up in `Infrastructure` where the HTTP context is available. This is also why an EF **interceptor** was chosen over overriding `DataContext.SaveChangesAsync` — it needs no constructor changes to a `DbContext` with 884 `DbSet` s and a parameterless constructor used by the migration tooling.

5.3 The recording rule

ONE RULE GOVERNS EVERYTHING

Log what you cannot recover from the live table.

Operation	What is written	Why
Modified	One row per genuinely changed field	The <i>old</i> value is about to be lost forever
Deleted	A <code>*DELETE*</code> marker plus a full field snapshot	The entire row is about to vanish
Added	A single <code>*CREATE*</code> marker	The values are still readable in the table itself — a column-by-column dump would be pure volume

5.4 The core of the interceptor

```
case EntityState.Modified:
    foreach (var prop in entry.Properties)
    {
        if (!prop.IsModified) continue;
        if (AuditedEntities.IsIgnoredField(prop.Metadata.Name)) continue;

        var oldValue = Format(prop.OriginalValue);
        var newValue = Format(prop.CurrentValue);

        // Handlers that do `Entry(x).State = Modified` mark every property
        // dirty whether or not it changed, so compare before emitting.
        if (string.Equals(oldValue, newValue, StringComparison.Ordinal)) continue;

        rows.Add(NewRow(entityName, prop.Metadata.Name, pkText,
            oldValue, newValue, now, changedBy, correlationId));
    }
    break;
```

WHY THAT COMPARISON MATTERS

Several handlers in this codebase set `Entry(x).State = EntityState.Modified`, which marks every column dirty regardless of whether it changed. Without the old-vs-new comparison, editing one field on a payment would have written ~30 rows instead of 1. Live testing confirmed it writes exactly 1.

5.5 The whitelist

The model has roughly 868 entities carrying `LastUpdatedStamp`. Auditing all of them field-by-field would bury the signal and grow the table faster than anyone could read it. Instead a deliberate whitelist — one file, one place to edit. It now holds **22 entities in two groups**.

Transactional — what happened

```
"OrderHeader", "OrderItem", "OrderItemBilling",    // order to cash
"Invoice", "InvoiceItem",                        // billing
"Payment", "PaymentApplication",                 // money movement
"AcctgTran", "AcctgTransEntry",                  // general ledger
"WorkEffort",                                    // projects
"SalesCommission", "SalesRequest",               // commissions
"InventoryItem", "InventoryItemDetail"           // stock
```

Master and reference data — how it is interpreted

```
"Party", "PartyGlAccount",                        // payroll + GL
attribution
"GlAccount", "GlSubClass", "GlSubClass2", "GlAccountCourseLabel", // GL classification
"CustomTimePeriod",                             // accounting periods
"InvoiceAttribute"                               // payroll method
snapshot
```

WHY THE SECOND GROUP EXISTS

The first group records *what happened*. The second records changes to the settings that decide *how it is interpreted* — and in several past incidents a field edited today silently changed what an already-closed period reported. That is a harder class of problem than a bad transaction, because nothing in the transaction itself looks wrong.

The second group was not guessed at. It came from reviewing the actual incident history of this system, and each entry earns its place:

Entity	The incident behind it
Party	Two fields with outsized reach. <code>PreferredPayrollPaymentMethodId</code> was flipped for one employee, which retroactively removed him from an <i>already-executed</i> payroll run — a payment of 322,078 could then no longer post against a re-derived run total of 262,745. <code>GlAccountIdAdvancedPayment</code> decides which GL account payroll debits, and is why a large share of payroll never reaches the P&L.
PartyGlAccount	The accrued ACCOUNTS_PAYABLE that payroll posting resolves against — directly in the path of the fix for the same incident.
GlAccount and the classification hierarchy	Roughly 210 manual <code>UPDATE</code> statements exist against <code>GL_ACCOUNT</code> in <code>API/Sql</code> — more than every other table combined. Classification drives the trial balance, every report and all of Power BI.
CustomTimePeriod	Six rows, rarely touched — but opening or closing a period changes what every balance inside it includes. Effectively free to audit.

InvoiceAttribute

Holds the `PAYROLL_PMT_METHOD` snapshot that exists *specifically* to stop payroll history being rewritten. Worth auditing the safety mechanism itself.

AN HONEST LIMIT ON THE GL ENTRIES

Most of those 210 `GL_ACCOUNT` corrections were made by running SQL by hand, which bypasses the application entirely — the change log will never see them. Auditing `GLAccount` captures edits made *through the app*, which is the case you cannot otherwise reconstruct. It is not a complete history of that table.

Four fields are always excluded (`LastUpdatedStamp` , `LastUpdatedTxStamp` , `CreatedStamp` , `CreatedTxStamp`) because they change on virtually every write and would roughly double the row count while adding nothing. Any field whose name contains *password*, *token*, *secret* or *apikey* is also skipped, and the same check redacts command payloads — so the two layers can never disagree about what counts as a secret.

5.6 Reusing the OFBiz columns

`ENTITY_AUDIT_LOG` came with two free text columns that fit exactly what was needed, which is why this layer needed no schema change:

Column	Now holds
CHANGED_BY_INFO	<code>Emad (3bb4e859-1157-4cc7-81b5-10f419359a41)</code> — name <i>and</i> ID, because display names can change later
CHANGED_SESSION_INFO	The correlation ID

6. Layer 2 — the activity log

Records *what the user asked for*, into a new `AUDIT_ACTIVITY` table.

6.1 Where the code lives

File	Role
Domain/AuditActivity.cs	The entity
Application/Core/AuditBehavior.cs	The MediatR pipeline behavior
Application/Core/AuditActivityWriter.cs	Writes on its own DbContext
Application/Core/IAuditableRequest.cs	Opt-in marker for non-command requests

6.2 Commands only

Of roughly 610 request types, about two thirds are queries that change nothing. Auditing them would multiply volume for almost no diagnostic value. The filter uses the codebase's own naming convention:

```
private static bool ComputeIsAudited()
{
    var type = typeof(TRequest);

    if (typeof(IAuditableRequest).IsAssignableFrom(type)) return true;

    return type.Name.Equals("Command", StringComparison.Ordinal)
        || type.Name.EndsWith("Command", StringComparison.Ordinal);
}
```

That matches the 218 nested `Command` classes plus a handful using a PascalCase suffix. The result is cached per closed generic type, so the reflection runs once per request type rather than on every dispatch.

6.3 Capturing the outcome correctly

Because handlers return failures rather than throwing, the behavior must unwrap the result — otherwise the trail would mark every single call a success:

```
var response = await next();

// Handlers return failed Results rather than throwing (see CLAUDE.md), so a failure
// looks like an ordinary return. Unwrap it or the trail would call every call a success.
if (response is IResult { IsSuccess: false } failed)
{
    activity.IsSuccess = false;
    activity.ErrorMessage = Truncate(failed.ErrorMessage, MaxErrorMessageLength);
}
else
{
    activity.IsSuccess = true;
}
```

Three distinct outcomes are recorded: success, a failed `Result` (`ExceptionType` stays null), and a thrown exception (`ExceptionType` populated, exception re-thrown unchanged).

6.4 The write happens in a `finally`

```
finally
{
    stopwatch.Stop();
    activity.DurationMs = (int)Math.Min(stopwatch.ElapsedMilliseconds, int.MaxValue);

    // Runs on the success path, the failed-Result path and the thrown-exception path.
    // The writer owns its own DataContext, so this row survives a rolled-back command.
    await _writer.WriteAsync(activity, cancellationToken);
}
```

The writer builds a short-lived `DataContext` from the DI-registered options and passes `CancellationToken.None` — when a client disconnects mid-request the record is still worth having. Any failure inside the writer is caught and logged: losing an audit row is survivable, failing a business request because auditing failed is not.

6.5 Payload capture and redaction

The command's input is serialized to JSON and stored. Registration and password-change commands travel through this same pipeline, so redaction is not optional — the payload is walked recursively and any property whose name looks like a credential is replaced with `***REDACTED***`. Serialization uses

`ReferenceHandler.IgnoreCycles` and `MaxDepth = 16`; commands carrying file streams that cannot be serialized store a marker instead of dropping the row.

7. Database changes

One migration, `20260826085724_AddAuditActivity`. It is purely additive — no drops, no data movement:

```
CREATE TABLE `AUDIT_ACTIVITY` (  
  `ACTIVITY_ID` varchar(36) NOT NULL,  
  `CORRELATION_ID` varchar(64) NULL,  
  `USER_NAME` varchar(255) NULL,  
  `USER_ID` varchar(36) NULL,  
  `REQUEST_NAME` varchar(255) NULL,  
  `REQUEST_PATH` varchar(512) NULL,  
  `HTTP_METHOD` varchar(10) NULL,  
  `CLIENT_IP_ADDRESS` varchar(64) NULL,  
  `REQUEST_JSON` text NULL,  
  `IS_SUCCESS` tinyint(1) NOT NULL,  
  `ERROR_MESSAGE` varchar(1024) NULL,  
  `EXCEPTION_TYPE` varchar(255) NULL,  
  `DURATION_MS` int NULL,  
  `STARTED_AT` datetime(6) NOT NULL,  
  `CREATED_STAMP` datetime NULL,  
  `LAST_UPDATED_STAMP` datetime NULL,  
  CONSTRAINT `PK_AUDIT_ACTIVITY` PRIMARY KEY (`ACTIVITY_ID`)  
);  
  
CREATE INDEX `AUDIT_ACTIVITY_USER` ON `AUDIT_ACTIVITY` (`USER_ID`, `STARTED_AT`);  
CREATE INDEX `AUDIT_ACTIVITY_REQUEST` ON `AUDIT_ACTIVITY` (`REQUEST_NAME`,  
  `STARTED_AT`);  
CREATE INDEX `AUDIT_ACTIVITY_CORRELATION` ON `AUDIT_ACTIVITY` (`CORRELATION_ID`);  
CREATE INDEX `AUDIT_ACTIVITY_STARTED` ON `AUDIT_ACTIVITY` (`STARTED_AT`);  
  
CREATE INDEX `ENTITY_AUDIT_LOG_CORRELATION` ON `ENTITY_AUDIT_LOG`  
  (`CHANGED_SESSION_INFO`);  
CREATE INDEX `ENTITY_AUDIT_LOG_RECORD` ON `ENTITY_AUDIT_LOG`  
  (`CHANGED_ENTITY_NAME`, `PK_COMBINED_VALUE_TEXT`, `CHANGED_DATE`);
```

`STARTED_AT` uses `datetime(6)` so several actions within the same second still sort correctly. The two `ENTITY_AUDIT_LOG` indexes were added because the OFBiz schema shipped with indexes only on the transaction stamps — the per-record history lookup, the query the History tab runs most, would otherwise scan the whole table. Key length was checked: ~2,045 bytes against InnoDB's 3,072-byte limit on a `Dynamic` row format.

IMPORTANT OPERATIONAL NOTE

`API/Program.cs:194` calls `context.Database.MigrateAsync()` at startup. Deploying is applying the migration — there is no separate approval step.

8. Configuration and the kill switch

Both capture layers check a single flag before doing any work — the interceptor before touching the `ChangeTracker`, the behavior before serializing anything. With it off, the entire feature costs one boolean check per command.

`API/appsettings.Production.json`:

```
"Auditing": {
  "Enabled": false,
  "Retention": {
    "Enabled": true,
    "ActivityDays": 365,
    "ChangeLogDays": 730,
    "RunAtHourUtc": 2,
    "BatchSize": 5000
  }
}
```

`docker-compose.vm.yml` — an environment variable that *overrides* the appsettings value:

```
Auditing__Enabled: "${Auditing_Enabled:-false}"
```

WHY BOTH

The appsettings file is baked into the Docker image, so changing it means a rebuild and redeploy. The environment variable turns that into a restart — which matters most in the case where auditing is misbehaving in production and you want it off quickly.

Development defaults to on; production ships off.

9. Retention

Without a purge the feature is a slow leak: `ENTITY_AUDIT_LOG` grows with every field change on every whitelisted entity, forever. Adding a purge once the table holds tens of millions of rows is considerably more painful than having one from the start — which is why it exists *before* auditing is switched on in production rather than after.

`Infrastructure/Auditing/AuditRetentionService.cs` is a `BackgroundService` (the first in this codebase) that trims both tables daily.

9.1 Three deliberate choices

Choice	Reason
Deletes run as batched raw SQL	A tracked delete of a million rows would exhaust memory; raw SQL does not re-enter the audit interceptor; and <code>LIMIT</code> keeps each statement's lock window short so a purge cannot stall live traffic.
<code>keepDays <= 0</code> means "keep forever"	The failure mode of a retention job is data loss, so it fails toward keeping. A mistyped config value cannot empty an audit table.

Its own enable flag

Independent of `Auditing:Enabled` — capture may be off while an existing backlog still needs trimming.

It also runs once about five minutes after startup, not only at the scheduled hour: a container restarting more than once a day would otherwise never reach 02:00.

9.2 Making the purge visible

Serilog's floor is `Warning` and both sinks carry their own `Warning` restriction, so an `Information` log would have been discarded twice over. Rather than lowering the global level and dragging every other Info log into the main log, the purge gets a `MinimumLevel.Override` plus a dedicated sub-logger sink:

```
.MinimumLevel.Override("Infrastructure.Auditing", LogEventLevel.Information)
.WriteTo.Logger(lc => lc
    .Filter.ByIncludingOnly(Matching.FromSource("Infrastructure.Auditing"))
    .WriteTo.File(
        Path.Combine(logDirectory, "audit-retention.log"),
        LogEventLevel.Information,
        rollingInterval: RollingInterval.Day,
        retainedFileCountLimit: 30))
```

A job that deletes millions of rows should leave a record; it should not drag everything else with it.

10. The read UI

The data existing is not the same as anyone using it. Two surfaces make the trail reachable.

10.1 Endpoints

Two OData entity sets, following the codebase's existing list-view convention. Both return `IQueryable`, so filtering, sorting and paging are pushed into SQL and nothing is materialised:

File	Role
Application/Auditing/AuditActivityRecord.cs	Projection, <code>[Key] ActivityId</code>
Application/Auditing/EntityAuditLogRecord.cs	Projection, <code>[Key] AuditHistorySeqId</code>
Application/Auditing/ListAuditActivities.cs	CQRS query
Application/Auditing/ListEntityAuditLogs.cs	CQRS query
API/Controllers/Auditing/*RecordsController.cs	Two <code>BaseODataController<T></code> controllers

10.2 The Audit Trail screen

`client-app/src/features/auditing/dashboard/AuditTrailList.tsx` — a filterable, sortable, paged grid over `AUDIT_ACTIVITY`: timestamp, user, action, outcome, duration, path, error, IP.

THE PART THAT MATTERS

Rows **expand** to show the field-level changes that action produced, fetched by correlation ID. That is the pivot from "who pressed what" to "what actually moved" — the entire point of the two-layer design, reduced to one click. When an action produced no field changes, the expanded row shows the redacted request payload instead.

Durations of three seconds or more render in orange. That is what surfaced an 11-second GL posting during testing.

10.3 The per-record History tab

`client-app/src/features/auditing/components/RecordHistory.tsx` renders *field: old ← new, by whom, when* for a single business record, with `*CREATE*` and `*DELETE*` markers shown as readable lines rather than empty value pairs.

It is wired into the payment form, following that container's existing pattern — the Actions menu opens it in a `ModalContainer`, exactly as Transactions and Payment Applications already work:

```
<RecordHistory entityName="Payment" pkText={`PaymentId=${payment.paymentId}`} />
```

ONE DETAIL THAT MUST BE EXACT

`pkText` has to match `PK_COMBINED_VALUE_TEXT` byte-for-byte (`"<PkPropertyName>=<value>"`). Get it wrong and the tab renders "no changes recorded" rather than an error — a silent failure. The format was confirmed against a real row, not assumed.

Access is gated on the existing `Admin` role, reached from the signed-in user menu. Per the frontend conventions this is a UX gate, not a security boundary — there is no backend role enforcement anywhere in this codebase.

11. What the live testing proved

Tested against the running development server, driving the real Arabic UI in Chrome, with every result verified directly in MySQL.

Command	OK	Duration	Change rows	What it proved
UpdatePayment	yes	5679 ms	1	Field-level capture; unchanged fields correctly suppressed
UpdatePayment	no	211 ms	0	Failed <code>Result</code> captured with its message; no phantom change rows
SetPaymentStatus	yes	11137 ms	5	Full GL posting captured
UpdateGLAccount	yes	2925 ms	1	Master-data whitelist entries fire correctly

11.1 The correlation join, end to end

```
Activity : UpdatePayment.Command    by Emad    correlation 0HN03HFAAMIPS:00000005
Change   : Payment / PaymentId=017827 / PaymentRefNum : '' → AUDIT-TEST-1
          changed by Emad (3bb4e859-1157-4cc7-81b5-10f419359a41)
          correlation 0HN03HFAAMIPS:00000005
```

11.2 Volume — the number that matters

A real double-entry GL posting produced exactly **five rows**:

```
AcctgTran      AcctgTransId=20899                *CREATE*
AcctgTransEntry AcctgTransId=20899::AcctgTransEntrySeqId=1 *CREATE*
AcctgTransEntry AcctgTransId=20899::AcctgTransEntrySeqId=2 *CREATE*
Payment        PaymentId=017827 StatusId          PMNT_NOT_PAID → PMNT_SENT
Payment        PaymentId=017827 ApprovedByPartyId NULL → 26
```

Confirmed against the database: 1 new **ACCTG_TRANS**, 2 new **ACCTG_TRANS_ENTRY**. The cost is **N ledger lines** → **N+1 marker rows, plus one row per genuinely changed field**. A 40-line posting would cost roughly 45 rows.

11.3 The retention job

```
18:28:49 Audit retention active: AUDIT_ACTIVITY 365d, ENTITY_AUDIT_LOG 730d, daily at
02:00 UTC
18:33:50 Audit purge found nothing older than the retention window
18:33:50 Next audit purge in 10.4 hours
```

The first line alone proves the chain: it compiled, the hosted service registered and started, the config bound, and the dedicated sink works — an **Information** event reached its own file while the global floor stayed at **Warning**. The startup purge then fired exactly five minutes in and scheduled the next run.

11.4 Other results

- **Redaction**: zero payloads containing credential-like text without **REDACTED**.
- **SaveChanges count safety**: around 12 handlers branch on the row count returned by **SaveChangesAsync**, which audit rows inflate. Every one uses **> 0** or **<= 0**, and audit rows only ever appear alongside business rows — so a save can never go from zero to non-zero. Verified in code and then in practice.

A GAP THE TESTING CAUGHT BEFORE DEPLOYMENT

The first live test showed **UpdateSalesCommission** rewriting a commission with **zero** field-level rows. Not a bug — **SalesCommission** simply was not on the whitelist. The original list was drawn around order-to-cash and the ledger, while the entity at the centre of the problems actually being investigated was invisible. That discovery is what prompted the whitelist review in §5.5.

A PERFORMANCE FINDING THE FEATURE SURFACED ON ITS OWN

`SetPaymentStatus` took **11.1 seconds** for a two-line GL posting; `UpdatePayment` took 5.7 s on first hit then 211 ms. The pattern points to first-hit EF query compilation against the 66,000-line model, which would be a one-off per query shape per restart — but 11 s is long even for that, and the production VM runs on a single pinned CPU. Worth investigating; not yet diagnosed.

12. Known limitations

Limitation	Detail
Bulk operations bypass the change log	There are ~70 <code>ExecuteUpdate</code> / <code>ExecuteDelete</code> / raw-SQL call sites. These never touch the <code>ChangeTracker</code> , so the interceptor cannot see them — including <code>ResetPayment</code> and <code>DeletePayrollInvoices</code> , the destructive operations most worth recording. The activity layer still records that the command ran and its outcome, but not which rows changed. Open decision.
Error-path latency amplification	The writer awaits a database write inside a <code>finally</code> . If a command failed <i>because</i> the database is unhealthy, the audit write must also time out before the error returns — adding delay during an incident. A queued or fire-and-forget write would fix it.
Hand-run SQL is invisible	Corrections applied directly to the database — the normal way GL classification is fixed here — bypass the application and leave no audit trace.
No automated tests	Verified manually end to end; nothing guards against regression.

13. Status and what comes next

Capture, retention and the read UI are all built. The feature is safe to deploy in its current state because it ships dormant behind the flag.

Remaining

1. **Test the OFF path.** All testing so far ran with auditing on; production ships off. That exact configuration has not yet been exercised.
2. **Decide what to do about the bulk-operation blind spot** — accept and document, or convert the highest-risk delete sites to record an explicit audit row.
3. **Consider automated tests** around the interceptor's comparison logic, which is the part most likely to regress silently.

Useful queries

```
-- What has this user been doing?
SELECT STARTED_AT, USER_NAME, REQUEST_NAME, IS_SUCCESS, DURATION_MS, ERROR_MESSAGE
FROM AUDIT_ACTIVITY ORDER BY STARTED_AT DESC LIMIT 50;

-- History of one record
SELECT CHANGED_DATE, CHANGED_BY_INFO, CHANGED_FIELD_NAME, OLD_VALUE_TEXT, NEW_VALUE_TEXT
FROM ENTITY_AUDIT_LOG
WHERE CHANGED_ENTITY_NAME = 'Invoice' AND PK_COMBINED_VALUE_TEXT = 'InvoiceId=INV1362'
ORDER BY CHANGED_DATE;

-- From a reported error ID straight to what it changed
SELECT a.REQUEST_NAME, a.USER_NAME, e.CHANGED_ENTITY_NAME,
       e.PK_COMBINED_VALUE_TEXT, e.CHANGED_FIELD_NAME, e.OLD_VALUE_TEXT, e.NEW_VALUE_TEXT
FROM AUDIT_ACTIVITY a
LEFT JOIN ENTITY_AUDIT_LOG e ON e.CHANGED_SESSION_INFO = a.CORRELATION_ID
WHERE a.CORRELATION_ID = '<trace id from the error>';
```

14. File manifest

Backend

File	Status	Purpose
Domain/AuditActivity.cs	new	Activity entity
Application/Core/AuditBehavior.cs	new	MediatR pipeline behavior
Application/Core/AuditActivityWriter.cs	new	Non-transactional writer
Application/Core/IAuditActivityWriter.cs	new	Writer abstraction
Application/Core/IAuditableRequest.cs	new	Opt-in marker
Application/Auditing/AuditActivityRecord.cs	new	OData projection
Application/Auditing/EntityAuditLogRecord.cs	new	OData projection
Application/Auditing/ListAuditActivities.cs	new	OData query
Application/Auditing/ListEntityAuditLogs.cs	new	OData query
Persistence/Auditing/AuditSaveChangesInterceptor.cs	new	Change-log capture
Persistence/Auditing/AuditedEntities.cs	new	Whitelist and exclusions
Persistence/Auditing/IAuditMetadataProvider.cs	new	Actor / correlation abstraction
Infrastructure/Security/HttpAuditMetadataProvider.cs	new	HTTP implementation
Infrastructure/Auditing/AuditRetentionService.cs	new	Daily purge
Infrastructure/Auditing/AuditRetentionOptions.cs	new	Retention settings
API/Controllers/Auditing/AuditActivityRecordsController.cs	new	OData endpoint

API/Controllers/Auditing/EntityAuditLogRecordsController.cs	new	OData endpoint
Persistence/Migrations/20260826085724_AddAuditActivity.cs	new	Table and indexes
Persistence/DataContext.cs	modified	DbSet , mapping, extra indexes
API/Extensions/ApplicationServiceExtensions.cs	modified	DI registration
API/Program.cs	modified	OData sets, retention log sink
API/appsettings.Production.json	modified	Kill switch, retention
docker-compose.vm.yml	modified	Environment override

Frontend

File	Status	Purpose
features/auditing/dashboard/AuditTrailList.tsx	new	Audit Trail screen
features/auditing/components/RecordHistory.tsx	new	Per-record History
app/store/apis/auditing/auditApi.ts	new	RTK Query endpoints
app/models/auditing/auditActivity.ts	new	TS model
app/models/auditing/entityAuditLog.ts	new	TS model
app/store/configureStore.ts	modified	Store registration
app/router/Routes.tsx	modified	/auditTrail route
app/layout/SignedInMenu.tsx	modified	Admin menu entry
app/common/messages/ar.json	modified	27 Arabic labels
features/accounting/payment/form/PaymentForm.tsx	modified	History modal
features/accounting/payment/form/PaymentActions.tsx	modified	History menu item